Project name: Vigenère Cryptography and Covering all Cases Exercise

Module name: Login-v2 Module

Developer: Caleb Metz

Course: CEN 4078 Secure Software Development

Date: 02/27/2026

Table 1. Login Module Test report

Data	***Proof*** - ***may be copy of code snippet*** - ***may be screen captures*** - ***may be written proof but must have data not just prose*** - ***check it after you submit. Make sure it is readable.*** - ***How about you make a table of inputs and results in addition to the above?***
Create a cryptographer class in your login module	`public interface Cryptographer {` 6 usages 1 implementation Caleb Metz `public String encrypt(String alphaKey, String input);` 6 usages 1 implementation Caleb Metz `public String decrypt(String alphaKey, String input);` 3 usages 1 implementation Caleb Metz `public int encrypt(int numberKey, int input);` 1 usage 1 implementation Caleb Metz `public int decrypt(int numberKey, int input);` 1 usage 1 implementation Caleb Metz `}`

<table>
<tr><td>Create an encryption method based on the Vigenère algorithm</td><td>

```java
private String vigenere(String alphaKey, String input, boolean encryption) {  3 usages
    int keyLength = alphaKey.length();
    int keyIndex = 0;

    StringBuilder result = new StringBuilder();

    for (char character: input.toCharArray()){
        result.append(caesar(alphaKey.charAt((keyIndex++ % keyLength)), character, encryption));
    }

    return result.toString();
}

private char caesar(char key, char input, boolean encrypting) {  1 usage

    String keyMinMax = findMinMax(key);
    String inputMinMax = findMinMax(input);

    if (inputMinMax == null || keyMinMax == null) {
        return input;
    }

    char keyMin;
    char keyMax;
    int keyAlphabetSize;
    char inputMin;
    char inputMax;
    int inputAlphabetSize;

    keyMin = keyMinMax.charAt(0);
    keyMax = keyMinMax.charAt(1);

    inputMin = inputMinMax.charAt(0);
    inputMax = inputMinMax.charAt(1);

    inputAlphabetSize = inputMax - inputMin + 1; //Fix 0 based index issue

    int offset = key - keyMin;
    //Gemini provided this solution when I was refactoring for decryption.
    int direction;
    if (Character.isDigit(input)) {
        direction = encrypting ? -1 : 1;
        inputAlphabetSize = 10;
    } else {
        direction = encrypting ? 1 : -1;
    }

    int normalizedInput = input - inputMin;

    int desiredCharIndex = (normalizedInput + (direction * (offset % inputAlphabetSize)) + inputAlphabetSize) % inputAlphabetSize;

    return (char) (desiredCharIndex + inputMin);
}
```

</td></tr>
<tr><td>Create a decryption method based on the Vigenère algorithm</td><td>The caesar private method includes a boolean to inverse into decryption which flips the offset to include a decrypt method. Decrypt() and Encrypt() are the exact same thing except for the boolean being different</td></tr>
<tr><td>Create an encryption method based on the numerical Vigenère table</td><td>

```java
int offset = key - keyMin;
//Gemini provided this solution when I was refactoring for decryption.
int direction;
if (Character.isDigit(input)) {
    direction = encrypting ? -1 : 1;
    inputAlphabetSize = 10;
} else {
    direction = encrypting ? 1 : -1;
}
```

The way I wrote my code allowed a quick change when Gemini analyzed it. I was able to implement the numerical table rather easily.

</td></tr>
<tr><td>Create an encryption method based on the</td><td>ditto</td></tr>
</table>

numerical Vigenère table	
Validate the alphabetic and numerical Vigenère encryption	
Validate the alphabetic and numerical Vigenère decryption	
Create a default-password class in your login module	

```java
@Test  Caleb Metz
void testBasicAlphaEncrypt() {

    StringBuilder dummyStringBuilder = new StringBuilder();

    for (int i = 0; i < alphaKey.length(); i++) {
        dummyStringBuilder.append("A");
    }
    String dummyString1 = dummyStringBuilder.toString();

    dummyStringBuilder = new StringBuilder();
    for (int i = 0; i < alphaKey.length(); i++) {
        dummyStringBuilder.append("a");
    }
    String dummyString2 = dummyStringBuilder.toString();

    assertEquals(alphaKey, vigenere.encrypt(alphaKey, dummyString1),  message: "String of 'A's should return the key");
    assertEquals(alphaKey.toLowerCase(), vigenere.encrypt(alphaKey, dummyString2),  message: "String of 'a's should return the key");
    assertEquals(basicAlphaResult,vigenere.encrypt(alphaKey, basicAlpha),  message: "Encrypt basic string");

}
```

```java
@Test  Caleb Metz
void testNumberEncrypt() {
    assertEquals(basicNumberResult,vigenere.encrypt(numberKey, basicNumber),  message: "Encrypt number");
}
```

```java
@Test  Caleb Metz
void testAlphaDecrypt() {

    StringBuilder dummyStringBuilder = new StringBuilder();

    for (int i = 0; i < alphaKey.length(); i++) {
        dummyStringBuilder.append("A");
    }
    String dummyString1 = dummyStringBuilder.toString();

    assertEquals(dummyString1, vigenere.decrypt(alphaKey, alphaKey),  message: "Decrypting the key should return 'A's");
    assertEquals(basicAlpha,vigenere.decrypt(alphaKey, basicAlphaResult),  message: "Decrypt basic string");

}
```

```java
@Test  Caleb Metz
void testNumberDecrypt() {
    assertEquals(basicNumber,vigenere.decrypt(numberKey, basicNumberResult),  message: "Decrypt number");
}
}
```

```java
public class DefaultPassword {  2 usages  Caleb Metz *
    //I relied heavily on Gemini for this. I am bad at using the Random library
    Validation validator;  1 usage
    Random random = new Random();  6 usages

    private static final String UPPER = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";  2 usages
    private static final String LOWER = "abcdefghijklmnopqrstuvwxyz";  2 usages
    private static final String DIGITS = "0123456789";  2 usages

    public DefaultPassword(Validation validator) {  no usages  new *
        this.validator = new Validation();
    }

    private int[] bucketGenerator() {  1 usage  new *
        int length = random.nextInt( i: 5) + 8;

        int numUpper = 1;
        int numLower = 1;
        int numDigit = 1;

        int remaining = length - 3;
```

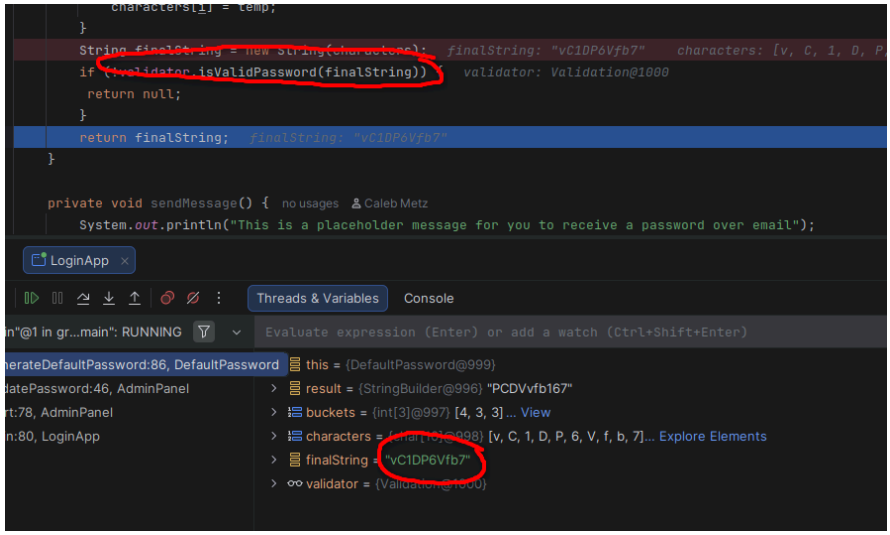
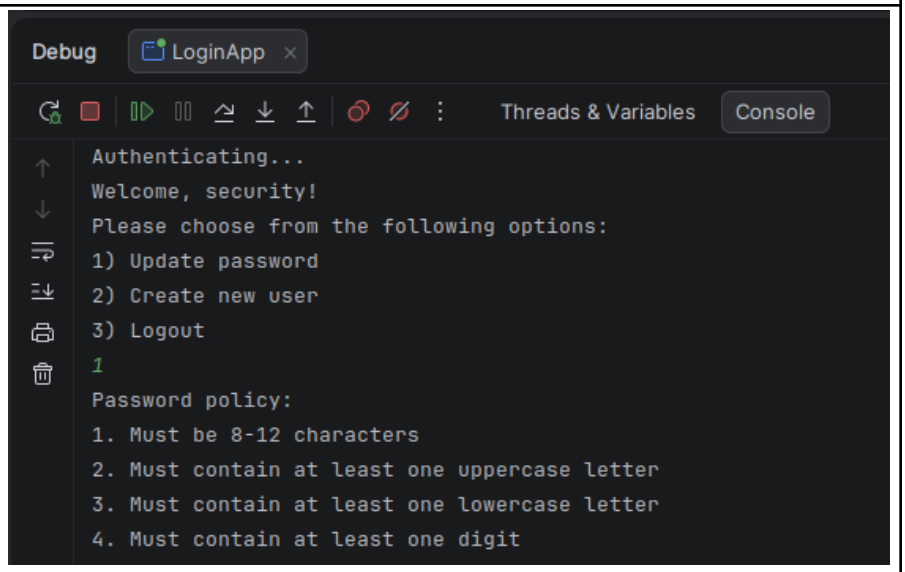
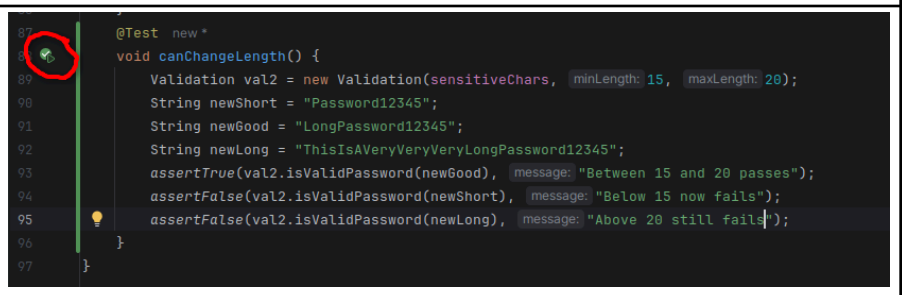
Create default-password method	
Create a method which returns the password policy.	

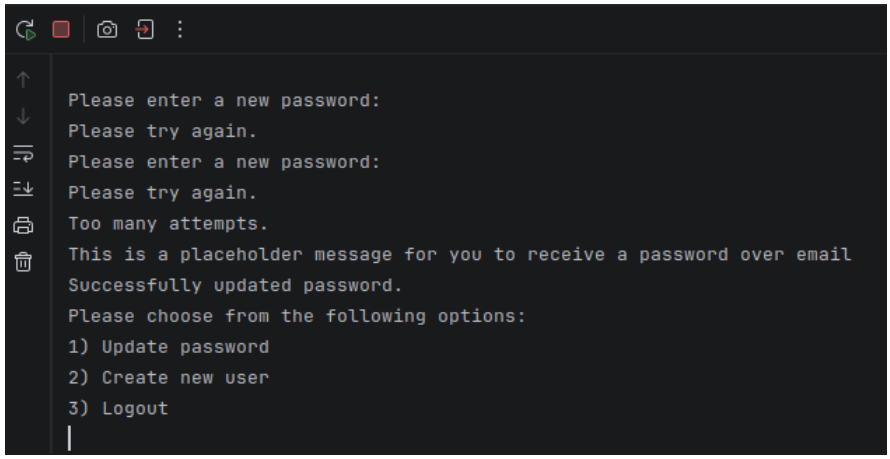
```java
public String generateDefaultPassword(Validation validator) {  no usages  new *

    StringBuilder result = new StringBuilder();

    int[] buckets = bucketGenerator();

    for (int i = 0; i < buckets[0]; i++) {
        result.append(UPPER.charAt(random.nextInt(UPPER.length())));
    }

    for (int i = 0; i < buckets[1]; i++) {
        result.append(LOWER.charAt(random.nextInt(LOWER.length())));
    }

    for (int i = 0; i < buckets[2]; i++) {
        result.append(DIGITS.charAt(random.nextInt(DIGITS.length())));
    }

    char[] characters = result.toString().toCharArray();
    for (int i = characters.length - 1; i > 0; i--) {
        int randomIndex = random.nextInt( i: i + 1);
        // Swap characters
        char temp = characters[randomIndex];
        characters[randomIndex] = characters[i];
        characters[i] = temp;
    }
    String finalString = new String(characters);
```

```java
//Added for V2
public String getPasswordPolicy() {  1 usage

    return "Password policy: \n" +
            "1. Must be " + minLength + "-" + maxLength + " characters\n" +
            "2. Must contain at least one uppercase letter\n" +
            "3. Must contain at least one lowercase letter\n" +
            "4. Must contain at least one digit\n";
}
public int getMinLength() {  no usages
    return minLength;
}

public int getMaxLength() {  no usages
    return maxLength;
}
```

Update your password validation code to user the method which returned the password policy.	

```java
public Validation(){  11 usages
    sensitiveChars = new char[]{'/', '-', ';', '"'};
    minLength = 8;
    maxLength = 12;
}

public Validation(char[] sensitiveChars, int minLength, int maxLength) {  7 usage
    this.sensitiveChars = sensitiveChars;
    this.minLength = minLength;
    this.maxLength = maxLength;
}

public boolean noSQLInjection(String input) {  4 usages  1 override

    if (input == null || input.isEmpty()) {
        return false;
    }

    for (char character : input.toCharArray()) {
        for (char special : sensitiveChars) {
            if (special == character) {
                return false;
            }
        }
    }
    return true;
}

public boolean isValidPassword(String input) {  12 usages
    //Length between 8-12 characters
    if (input.length() < minLength || input.length() > maxLength) {
        return false;
    }
    //Contains at least one upper [Cite: Gemini & IntelliJ]
    if (input.chars().noneMatch(Character::isUpperCase)) {
        return false;
    };
    //Contains at least one lower
    if (input.chars().noneMatch(Character::isLowerCase)) {
        return false;
    }
    //Contains at least one number
    if (input.chars().noneMatch(Character::isDigit)) {
        return false;
```

Validate the method which creates the default-password.	 Will fail if it does not meet policy. Debugger shows it is working.
Validate the method which returns the password policy.	
Validate the changes for testing passwords now that you must call the password policy.	

Validate the notice to the user about the default password creation and notification.	
Did you meet all the requirements. If not, describe why?	I believe so. I had to make minor changes in the name of good architecture like making Cryptographer an interface which Vigener implements. I also kind of eliminated the need for the number key since the passwords need the numbers encrypted too. It still encrypts numbers according to the table, it just uses either letters or numbers as a key making it true to the original still.
Did you make changes to code from past assignments? If so, please describe including what requirements were tied to the code.	Added cryptographer to auth service Removed debug print statements from LoginApp (previously showcased creating a user) Added a user creation prompt post login Updated tests as I went along Created an AdminPanel app which allows you to have a CLI experience Made Authenticate return a user, allows AdminPanel to easily accept authentication hardened authservice tests Allowed checking and saving a User object directly when creating users Added exit codes to clarify login failed Updated MockDB to test saving and logging in as the same user Fixed FileUserDB null point exception with findByUsername Added getMinLength and getMaxLength for Validation to allow defaultPassword to account for it